Interpreting SVM design using Scikit-Learn and results

Aldebaro Klautau
Federal University of Pará (UFPA) / LASSE

Computational Intelligence Class

2026

Designing SVMs with Scikit-Learn

Code: https://scikit-learn.org/stable/modules/svm.html

Classes: SVC, NuSVC (https://www.csie.ntu.edu.tw/~cjlin/libsvm) and LinearSVC (https://www.csie.ntu.edu.tw/~cjlin/liblinear)

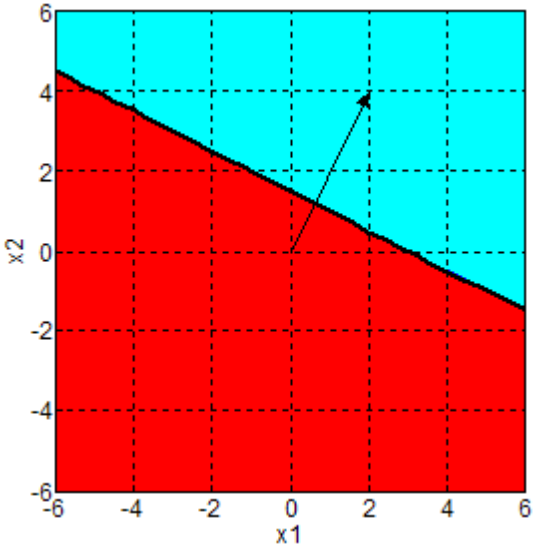
Examples:

```
linear_svc = svm.LinearSVC(C=4, max_iter=1e4, dual=True, tol=1e-10)
svc_with_linear_kernel = svm.SVC(kernel='linear', C=2, verbose=1, shrinking=False)
rbf_svm = svm.SVC(kernel='rbf', gamma=0.7, C=1) #RBF also called Gaussian
polynomial_svm = svm.SVC(kernel='poly', degree=3, gamma='auto', C=100, coef0=0)
```

Linear SVM as a perceptron

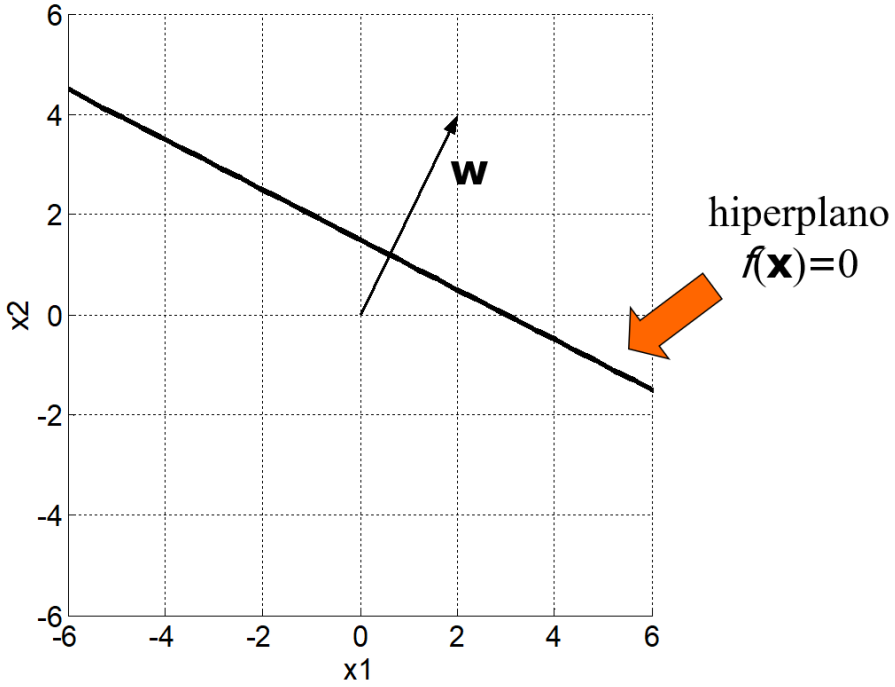
- $f(\mathbf{x})=\text{sgn}(<\mathbf{x}, \mathbf{w}> + b)$
  - $\mathbf{x},\mathbf{w} \in R^d$, $f(\mathbf{x}) \in \{-1, +1\}$ e $b \in R$ ("bias")
  - $<\mathbf{x},\mathbf{w}> = \|\mathbf{x}\| \, \|\mathbf{w}\| \cos\theta = \sum_{n=1}^{d} x_n w_n$ é o produto interno
  - sgn retorna o sinal, com sgn (0) = 1
  - **w** é um vetor normal ao *hiperplano separador*

Exemplo: d=2, $x=(x_1, x_2)$



Example of linear SVM

$f(\mathbf{x})=\text{sgn}(\langle\mathbf{x}, \mathbf{w}\rangle + b)$, com $\mathbf{w}=(2, 4)$, $b=-6$



Non-linear SVM

$$f(\mathbf{z}) = \left(\sum_{n=0}^{N-1} \lambda_n K(\mathbf{z}, \mathbf{x}_n) \right) + b$$

where:
N is the number of training examples
z is the input (test) vector
$\mathbf{x}_n$ is the n-th training example
b is the bias, also called "intercept"
λ_n is the dual coefficient of the n-th training example, multiplied by the sign of its label y_n
but sometimes called simply "dual coefficient"

When λ_n is not zero, the corresponding n-th training example is a support vector

Examples of decision regions for two linear and two non-linear SVMs

- Code ak_svm_prova1.py

Inputs:

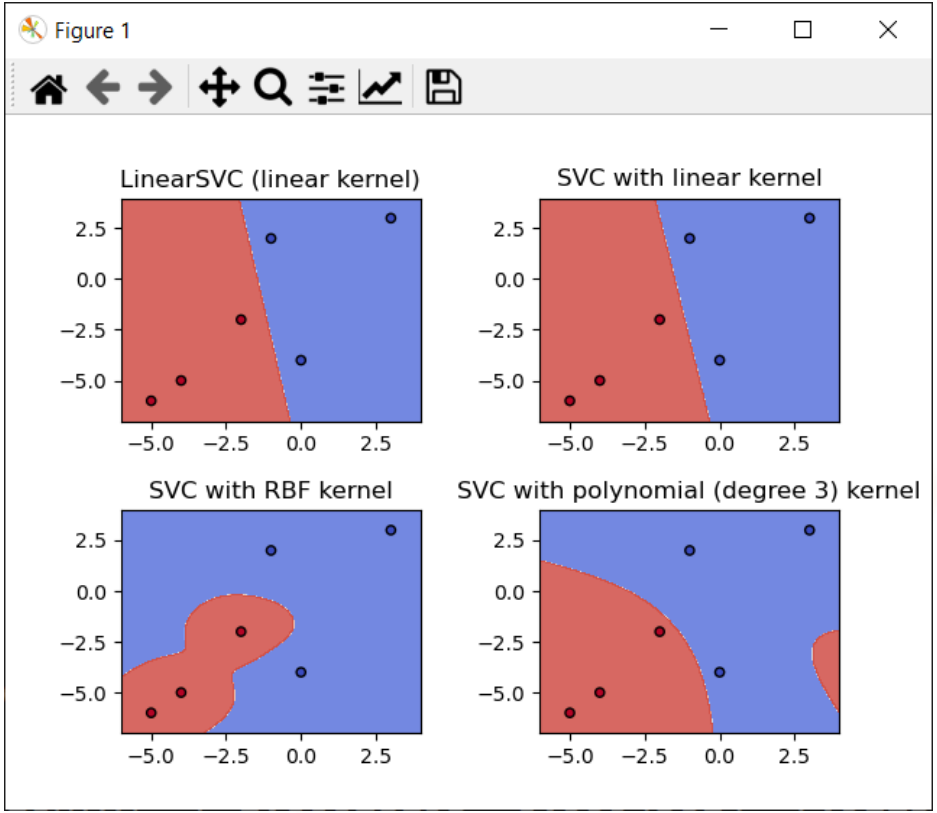
Normalized training set:

```
[[ 0. -4.]
 [-1.  2.]
 [ 3.  3.]
 [-5. -6.]
 [-4. -5.]
 [-2. -2.]]
```

Kernels:

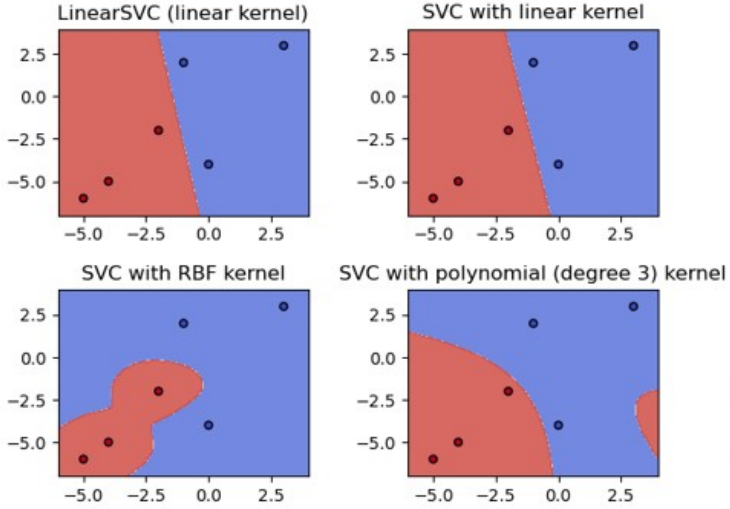
- linear: $\langle x, x' \rangle$
- polynomial: $(\gamma \langle x, x' \rangle + r)^d$
- rbf: $\exp(-\gamma \|x - x'\|^2)$

Output labels: first 3 Examples above have label 0 and the last 3 have label 1



Kernels (as implemented in sklearn)

- linear: $\langle x, x' \rangle$.
- polynomial: $(\gamma \langle x, x' \rangle + r)^d$, where d is specified by parameter `degree`, r by `coef0`.
- rbf: $\exp(-\gamma \|x - x'\|^2)$, where γ is specified by parameter `gamma`, must be greater than 0.
- sigmoid $\tanh(\gamma \langle x, x' \rangle + r)$, where r is specified by `coef0`.



Example of non-linear SVM with polynomial kernel of degree 3

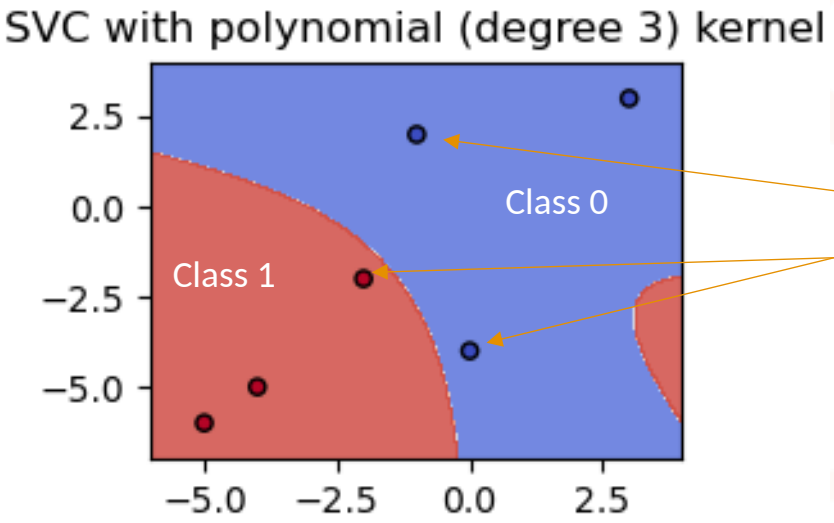
- Code ak_svm_prova1.py

Total of 3 support vectors: 2 for class 0 and 1 for class 1

Inputs:

Normalized training set:

[[0. -4.]
[-1. 2.]
[3. 3.]
[-5. -6.]
[-4. -5.]
[-2. -2.]]



svm.n_support_= [2 1]
svm.support_= [0 1 5]
svm.support_vectors_= [[0. -4.]
[-1. 2.]
[-2. -2.]]

Output labels: first 3 Examples above have label 0 and the last 3 have label 1

Output of the SVC class for the SVM with polynomial kernel of degree 3 in previous slide

polynomial: $(\gamma\langle x, x'\rangle + r)^d$, where d is specified by parameter `degree`, r by `coef0`

4) SVM with SVC

{'C': 1, 'break_ties': False, 'cache_size': 200, 'class_weight': None, 'coef0': 0, 'decision_function_shape': 'ovr', 'degree': 3, 'gamma': 'auto', 'kernel': 'poly', 'max_iter': -1, 'probability': False, 'random_state': None, 'shrinking': True, 'tol': 0.001, 'verbose': False}

svm.n_support_= [2 1]

svm.support_= [0 1 5]

svm.support_vectors_= [[0. -4.]

[-1. 2.]

[-2. -2.]]

svm.dual_coef_= [[-0.00887134 -0.03133903 0.04021037]]

svc.intercept_= [-1.03731897]

At most 10 decisions: svm.decision_function(X)= [-1.00028193 -0.99943614 -7.91231897 38.49667194 20.25085641 0.99971807]

Accuracy via svm.score(X,y)= 1.0

Gamma= 0.5

Example of non-linear SVM with RBF kernel

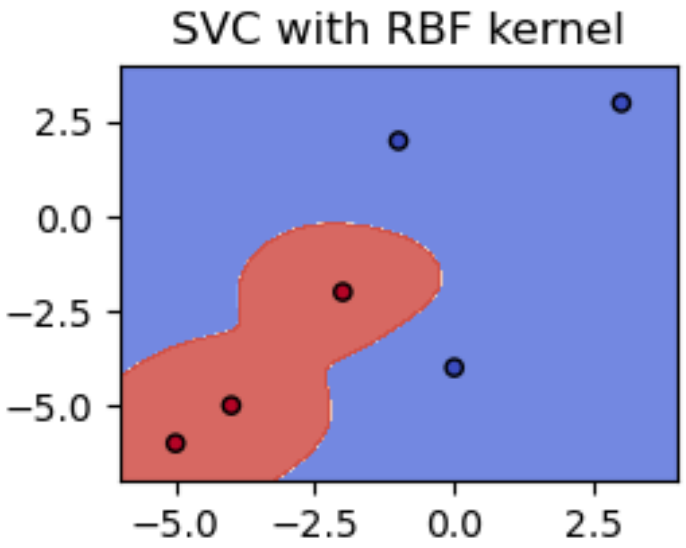
- Code ak_svm_prova1.py

Total of 6 support vectors (SVs): 3 for class 0 and 3 for class 1. In this case, all training examples are SVs

Inputs:

Normalized training set:

```
[[ 0. -4.]
 [-1.  2.]
 [ 3.  3.]
 [-5. -6.]
 [-4. -5.]
 [-2. -2.]]
```




```
svm.n_support_= [3 3]
svm.support_= [0 1 2 3 4 5]
svm.support_vectors_= [[ 0. -4.]
 [-1.  2.]
 [ 3.  3.]
 [-5. -6.]
 [-4. -5.]
 [-2. -2.]]
```

Output labels: first 3 Examples above have label 0 and the last 3 have label 1

Output of the SVC class for the RBF SVM in previou

rbf: $\exp(-\gamma\|x - x'\|^2)$

3) SVM with SVC

{'C': 1, 'break_ties': False, 'cache_size': 200, 'class_weight': None, 'coef0': 0.0, 'decision_function_shape': 'ovr', 'degree': 3, 'gamma': 0.7, 'kernel': 'rbf', 'max_iter': -1, 'probability': False, 'random_state': None, 'shrinking': True, 'tol': 0.001, 'verbose': False}

svm.n_support_= [3 3]

svm.support_= [0 1 2 3 4 5]

svm.support_vectors_= [[0. -4.]

[-1. 2.]

[3. 3.]

[-5. -6.]

[-4. -5.]

[-2. -2.]]

svm.dual_coef_= [[-0.91722233 -0.91351914 -0.91300432 0.87185969 0.8718861   1.   ]]

svc.intercept_= [-0.08676121]

At most 10 decisions: svm.decision_function(X)= [-1.00027976 -1.00027976 -0.99977173 1.00010297 1.00022828 0.90993821]

Accuracy via svm.score(X,y)= 1.0

Gamma= 0.7

Writing the SVM decision function after training the model (example 4: polynomial)

polynomial: $(\gamma\langle x, x'\rangle + r)^d$, where d is specified by parameter `degree`, r by `coef0`

4) SVM with SVC

{'C': 1, 'break_ties': False, 'cache_size': 200, 'class_weight': None, 'coef0': 0, 'decision_function_shape': 'ovr', 'degree': 3, 'gamma': 'auto', 'kernel': 'poly', 'max_iter': -1, 'probability': False, 'random_state': None, 'shrinking': True, 'tol': 0.001, 'verbose': False}

svm.n_support_= [2 1]

svm.support_= [0 1 5]

svm.support_vectors_= [[0. -4.]

[-1. 2.]

[-2. -2.]]

svm.dual_coef_= [[-0.00887134 -0.03133903 0.04021037]]

svc.intercept_= [-1.03731897]

Gamma= 0.5

Writing the SVM decision function after training the model (example 3: RBF)

$$\text{rbf: } \exp(-\gamma \|x - x'\|^2)$$

3) SVM with SVC

{'C': 1, 'break_ties': False, 'cache_size': 200, 'class_weight': None, 'coef0': 0.0, 'decision_function_shape': 'ovr', 'degree': 3, 'gamma': 0.7, 'kernel': 'rbf', 'max_iter': -1, 'probability': False, 'random_state': None, 'shrinking': True, 'tol': 0.001, 'verbose': False}

svm.n_support_= [3 3]

svm.support_= [0 1 2 3 4 5]

svm.support_vectors_= [[0. -4.]

[-1. 2.]

[3. 3.]

[-5. -6.]

[-4. -5.]

[-2. -2.]]

svm.dual_coef_= [[-0.91722233 -0.91351914 -0.91300432 0.87185969 0.8718861 1.]]

svc.intercept_= [-0.08676121]

Gamma= 0.7

Recall the dual coefficient in non-linear SVMs

$$f(\mathbf{z}) = \left(\sum_{n=0}^{N-1} \lambda_n K(\mathbf{z}, \mathbf{x}_n) \right) + b$$

where:
N is the number of training examples
z is the input (test) vector
$\mathbf{x}_n$ is the n-th training example
b is the bias, also called "intercept"
λ_n is the dual coefficient of the n-th training example, multiplied by the sign of its label y_n
but sometimes called simply "dual coefficient"

When λ_n is not zero, the corresponding n-th training example is a support vector

From https://scikit-learn.org/stable/modules/svm.html:

The dual problem to the primal is

$$\begin{aligned} \min_{\alpha} \frac{1}{2}\alpha^T Q\alpha - e^T\alpha \\ \text{subject to } y^T\alpha = 0 \\ 0 \leq \alpha_i \leq C, i = 1, \ldots, n \end{aligned}$$

where e is the vector of all ones, and Q is an n by n positive semidefinite matrix, $Q_{ij} \equiv y_i y_j K(x_i, x_j)$, where $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ is the kernel. The terms α_i are called the dual coefficients, and they are upper-bounded by C. This dual representation highlights the fact that training vectors are implicitly mapped into a higher (maybe infinite) dimensional space by the function ϕ: see kernel trick.

Once the optimization problem is solved, the output of decision_function for a given sample x becomes:

$$\sum_{i \in SV} y_i \alpha_i K(x_i, x) + b,$$

and the predicted class correspond to its sign. We only need to sum over the support vectors (i.e. the samples that lie within the margin) because the dual coefficients α_i are zero for the other samples.

These parameters can be accessed through the attributes `dual_coef_` which holds the product $y_i \alpha_i$, `support_vectors_` which holds the support vectors, and `intercept_` which holds the independent term b

Converting linear SVM into a perceptron

- We want to write as $f(\mathbf{z})=\mathrm{sgn}(\langle \mathbf{z}, \mathbf{w} \rangle + b)$ instead of writing as:

$$f(\mathbf{z}) = \left(\sum_{n=0}^{N-1} \lambda_n K(\mathbf{z}, \mathbf{x}_n) \right) + b$$

For that, we want to understand the code:

```python
def convert_linear_SVM_to_perceptron(support_vectors, dual_coef):
    dual_coef = np.ravel(dual_coef,order='C') #convert to a 1D vector
    num_support_vectors = len(dual_coef)
    if support_vectors.shape[0] != num_support_vectors:
        raise Exception('support_vectors.shape[0] != num_support_vectors')
    input_space_dimension = support_vectors.shape[1]
    perceptron_weights = np.zeros((input_space_dimension))
    for sv in range(num_support_vectors):
        perceptron_weights += dual_coef[sv] * support_vectors[sv]
    return perceptron_weights
```

Example of linear SVM as perceptron

2) SVM with SVC

{'C': 1, 'break_ties': False, 'cache_size': 200, 'class_weight': None, 'coef0': 0.0, 'decision_function_shape': 'ovr', 'degree': 3, 'gamma': 'scale', 'kernel': 'linear', 'max_iter': -1, 'probability': False, 'random_state': None, 'shrinking': False, 'tol': 0.001, 'verbose': 1}

svm.n_support_= [2 1]

svm.support_= [0 1 5]

svm.support_vectors_= [[0. -4.]

[-1. 2.]

[-2. -2.]]

svm.dual_coef_= [[-0.45994152 -0.27992202 0.73986354]]

svc.intercept_= [-1.79954513]

Gamma= 0.053452115812917596

svc_with_linear_kernel.coef_= [[-1.19980506 -0.19980506]]

Estimated perceptron_weights= [-1.19980506 -0.19980506]

Estimated bias= -1.7997075844389612

Converting linear SVM into a perceptron

A forma geral de uma SVM obtida pela solução do problema dual é

$$f(z) = \left(\sum_{n=0}^{N-1} \lambda_n K(z, x_n) \right) + b \qquad \text{(Eq. 1)}$$

onde x_n é o n-ésimo exemplo dentre os N exemplos do conjunto de treino. Quando o kernel $K(z, x_n)$ é linear, ou seja, o produto interno $K(z, x_n) = \langle z, x_n \rangle$, por linearidade é possível simplificar para:

$$f(z) = \left(\sum_{n=0}^{N-1} \lambda_n \langle z, x_n \rangle \right) + b = \left\langle z, \sum_{n=0}^{N-1} \lambda_n x_n \right\rangle + b \qquad \text{(Eq. 2)}$$

onde $w = \sum_{n=0}^{N-1} \lambda_n x_n$ é o vetor de pesos do perceptron equivalente. Um exemplo ajuda a esclarecer. Um espaço de entrada é D=2 e

- In summary: we multiply the n-th training example by its corresponding lambda value

Converting linear SVM into a perceptron

- In summary: we multiply the n-th training example by its corresponding lambda value

```python
def convert_linear_SVM_to_perceptron(support_vectors, dual_coef):
    dual_coef = np.ravel(dual_coef,order='C') #convert to a 1D vector
    num_support_vectors = len(dual_coef)
    if support_vectors.shape[0] != num_support_vectors:
        raise Exception('support_vectors.shape[0] != num_support_vectors')
    input_space_dimension = support_vectors.shape[1]
    perceptron_weights = np.zeros((input_space_dimension))
    for sv in range(num_support_vectors):
        perceptron_weights += dual_coef[sv] * support_vectors[sv]
    return perceptron_weights
```

Example of linear SVM as perceptron

2) SVM with SVC

{'C': 1, 'break_ties': False, 'cache_size': 200, 'class_weight': None, 'coef0': 0.0, 'decision_function_shape': 'ovr', 'degree': 3, 'gamma': 'scale', 'kernel': 'linear', 'max_iter': -1, 'probability': False, 'random_state': None, 'shrinking': False, 'tol': 0.001, 'verbose': 1}

svm.n_support_= [2 1]

svm.support_= [0 1 5]

svm.support_vectors_= [[0. -4.]

[-1. 2.]

[-2. -2.]]

svm.dual_coef_= [[-0.45994152 -0.27992202 0.73986354]]

svc.intercept_= [-1.79954513]

Gamma= 0.053452115812917596

svc_with_linear_kernel.coef_= [[-1.19980506 -0.19980506]]

Estimated perceptron_weights= [-1.19980506 -0.19980506]

Estimated bias= -1.7997075844389612